ระบบ User Module (รวม Middleware JWT)

URL Local: http://localhost:5000

สารบัญ

1. โครงสร้างโฟลเดอร์
2. หลักการ (Concept)
3. ไฟล์และโค้ดทั้งหมด (Full Code)
    - 3.1 Middleware: `middleware/jwtauth.go`
    - 3.2 User Module Interfaces
    - 3.3 Delivery (HTTP)
    - 3.4 Usecase
    - 3.5 Repository
    - 3.6 Distributor & Processor (Async Tasks)
    - 3.7 Presenter (DTO)
4. วิธีการเพิ่มฟังก์ชันหรือแก้ไข (Extension Guide)
    - 4.1 ขั้นตอนทั่วไปในการเพิ่ม endpoint ใหม่
    - 4.2 การเพิ่ม OTP (One-Time Password) โดยไม่เปลี่ยนโครงสร้างเดิม
    - 4.3 การเพิ่ม 2FA (TOTP) แบบแยกตาราง
    - 4.4 การเพิ่ม Social Login (Google/Facebook)
    - 4.5 การเพิ่ม Rate Limiting
    - 4.6 การเปลี่ยนจาก Asynq ไปใช้ระบบอื่น
5. Checklist ทดสอบ

---

1. โครงสร้างโฟลเดอร์ (Folder Structure)

```
internal/
│
├── middleware/                          #  middleware ระดับโปรเจกต์
│   └── jwtauth.go                       #  JWT verification + authenticator +
current user
│
└── user/                                #  root module ของ user
    │
    ├── users/                           #  (actual module name)
    │   ├── delivery/
    │   │   └── http/
    │   │       ├── handlers.go          #  HTTP handlers (Register, Me, Update,
etc.)
    │   │       └── routes.go            #  Route registration
    │   ├── distributor/
    │   │   └── distributor.go           #  Asynq task distributor (send email)
    │   ├── presenter/
    │   │   └── presenters.go            #  Request/Response DTOs
```

```
    │   ├── processor/
    │   │   └── processor.go            #  Asynq task processor (consume email
tasks)
    │   ├── repository/
    │   │   ├── pg_repository.go        #  PostgreSQL implementation
    │   │   └── redis_repository.go     #  Redis implementation (cache + refresh
tokens)
    │   └── usecase/
    │       └── usecase.go              #  Business logic
    │
    ├── handler.go                      #  Interface of HTTP handlers
    ├── pg_repository.go                #  Interface of PostgreSQL repository
    ├── redis_repository.go             #  Interface of Redis repository
    ├── usecase.go                      #  Interface of UseCase
    └── worker.go                       #  Async task definitions
```

---

2. หลักการ (Concept)

คืออะไร?

ระบบ User Module จัดการผู้ใช้ด้วย Clean Architecture แยกชั้น Delivery, UseCase, Repository ช่วยให้ทดสอบง่าย เปลี่ยนเทคโนโลยีได้สะดวก ใช้ JWT แบบ RS256, Redis สำหรับ cache และ refresh token, และ Asynq สำหรับส่งอีเมล async

มีกี่แบบ?

Pattern	คำอธิบาย
Clean Architecture	แยกชั้นชัดเจน, inversion
Repository Pattern	ซ่อนการเข้าถึง PostgreSQL/Redis ไว้เบื้องหลัง interface
Asynchronous Task Queue	ใช้ Asynq + Redis ส่งอีเมลไม่ block response
JWT with RS256	ใช้ private/public key เซ็น token, รองรับ refresh token rotation
Redis Caching	cache user data และเก็บ refresh token lists (set)

ข้อห้ามสำคัญ

- **ห้ามเก็บ plaintext password** – ต้อง bcrypt เสมอ
- **ห้ามส่ง sensitive data** (password, verification_code, reset_token) ใน JSON response
- **ห้ามใช้ Access Token นานเกินไป** (ตั้ง 15-30 นาที)
- **ห้ามให้ OTP endpoint โดยไม่มี rate limiting** (เสี่ยง brute force)

---

3. ไฟล์และโค้ดทั้งหมด (Full Code)

3.1 Middleware: `middleware/jwtauth.go`

```go
package middleware

import (
    "context"
    "errors"
    "net/http"
    "strings"

    "gorestapi/internal/models"
    "gorestapi/pkg/httpErrors"
    "gorestapi/pkg/jwt"
    "gorestapi/pkg/responses"

    "github.com/go-chi/render"
    "github.com/google/uuid"
)

var (
    TokenCtxKey = &contextKey{"Token"}
    IdCtxKey    = &contextKey{"Id"}
    EmailCtxKey = &contextKey{"Email"}
    ErrorCtxKey = &contextKey{"Error"}
    UserCtxKey  = &contextKey{"User"}
)

type contextKey struct {
    name string
}

func (k *contextKey) String() string {
    return "jwtauth context value " + k.name
}

// Verifier ดึง JWT จาก header หรือ cookie แล้ว set ลง context
// Verifier extracts JWT from header/cookie and sets into context
func (mw *MiddlewareManager) Verifier(requireAccessToken bool) func(http.Handler)
http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            ctx := r.Context()

            token := TokenFromHeader(r)

            if token == "" {
                err := httpErrors.ErrTokenNotFound(errors.New("not found token in
header"))
                ctx = context.WithValue(ctx, ErrorCtxKey, err)
            } else {
                var publicKey string
                if requireAccessToken {
                    publicKey = mw.cfg.Jwt.AccessTokenPublicKey
                } else {
                    publicKey = mw.cfg.Jwt.RefreshTokenPublicKey
```

```go
                }
                id, email, err := jwt.ParseTokenRS256(token, publicKey)
                ctx = context.WithValue(ctx, TokenCtxKey, token)
                ctx = context.WithValue(ctx, IdCtxKey, id)
                ctx = context.WithValue(ctx, EmailCtxKey, email)
                ctx = context.WithValue(ctx, ErrorCtxKey, err)
            }

            next.ServeHTTP(w, r.WithContext(ctx))
        })
    }
}

// Authenticator ตรวจสอบว่ามี error จาก Verifier หรือไม่ ถ้ามี => 401
// Authenticator checks if Verifier set an error -> 401 Unauthorized
func (mw *MiddlewareManager) Authenticator() func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            err, _ := r.Context().Value(ErrorCtxKey).(error)
            if err != nil {
                render.Render(w, r, responses.CreateErrorResponse(err))
                return
            }
            next.ServeHTTP(w, r)
        })
    }
}

// CurrentUser โหลด user object จาก DB/Redis และใส่ใน context
// CurrentUser loads user object from DB/Redis and stores in context
func (mw *MiddlewareManager) CurrentUser() func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            ctx := r.Context()

            id, _ := r.Context().Value(IdCtxKey).(string)
            err, _ := r.Context().Value(ErrorCtxKey).(error)

            if err != nil || id == "" {
                render.Render(w, r,
responses.CreateErrorResponse(httpErrors.ParseErrors(err)))
                return
            }

            idParsed, err := uuid.Parse(id)
            if err != nil {
                render.Render(w, r,
responses.CreateErrorResponse(httpErrors.ErrInvalidJWTClaims(errors.New("can not
convert id to uuid from id in token"))))
                return
            }

            user, err := mw.usersUC.Get(ctx, idParsed)
            if err != nil {
```

```go
                render.Render(w, r, responses.CreateErrorResponse(err))
                return
            }

            ctx = context.WithValue(ctx, UserCtxKey, user)
            next.ServeHTTP(w, r.WithContext(ctx))
        })
    }
}

// SuperUser ตรวจสอบว่า user มีสิทธิ์ superuser หรือไม่
// SuperUser checks if user has superuser privileges
func (mw *MiddlewareManager) SuperUser() func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            ctx := r.Context()
            user, err := GetUserFromCtx(ctx)
            if err != nil {
                render.Render(w, r, responses.CreateErrorResponse(err))
                return
            }
            if !mw.usersUC.IsSuper(ctx, *user) {
                render.Render(w, r,
responses.CreateErrorResponse(httpErrors.ErrNotEnoughPrivileges(errors.New("user
is not super user"))))
                return
            }
            next.ServeHTTP(w, r)
        })
    }
}

// ActiveUser ตรวจสอบว่า user status = active
// ActiveUser checks if user status is active
func (mw *MiddlewareManager) ActiveUser() func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            ctx := r.Context()
            user, err := GetUserFromCtx(ctx)
            if err != nil {
                render.Render(w, r, responses.CreateErrorResponse(err))
                return
            }
            if !mw.usersUC.IsActive(ctx, *user) {
                render.Render(w, r,
responses.CreateErrorResponse(httpErrors.ErrInactiveUser(errors.New("user
inactive"))))
                return
            }
            next.ServeHTTP(w, r)
        })
    }
}
```

```go
// TokenFromHeader ดึง token จาก Authorization header
// TokenFromHeader extracts token from Authorization header
func TokenFromHeader(r *http.Request) string {
    bearer := r.Header.Get("Authorization")
    if len(bearer) > 7 && strings.ToUpper(bearer[0:6]) == "BEARER" {
        return bearer[7:]
    }
    return ""
}

// GetUserFromCtx ดึง user object จาก context
// GetUserFromCtx retrieves user object from context
func GetUserFromCtx(ctx context.Context) (*models.User, error) {
    user, ok := ctx.Value(UserCtxKey).(*models.User)
    if !ok {
        return nil, httpErrors.ErrUnauthorized(errors.New("can convert user from
context"))
    }
    return user, nil
}
```

3.2 User Module Interfaces

`handler.go` **(HTTP handlers interface)**

```go
package users

import "net/http"

type Handlers interface {
    // Public
    Register() http.HandlerFunc

    // Authenticated
    Me() http.HandlerFunc
    UpdateMe() http.HandlerFunc
    UpdatePasswordMe() http.HandlerFunc

    // Admin
    Create() http.HandlerFunc
    GetMulti() http.HandlerFunc
    Get() http.HandlerFunc
    Update() http.HandlerFunc
    UpdatePassword() http.HandlerFunc
    Delete() http.HandlerFunc
    UpdateRole() http.HandlerFunc
    LogoutAllAdmin() http.HandlerFunc
}
```

`pg_repository.go` **(PostgreSQL repository interface)**

```go
package users

import (
    "context"
    "time"

    "icmongolang/internal"
    "icmongolang/internal/models"
)

type UserPgRepository interface {
    internal.PgRepository[models.SdUser]
    GetByEmail(ctx context.Context, email string) (*models.SdUser, error)
    UpdatePassword(ctx context.Context, exp *models.SdUser, newPassword string)
(*models.SdUser, error)
    UpdateVerificationCode(ctx context.Context, exp *models.SdUser,
newVerificationCode string) (*models.SdUser, error)
    UpdateVerification(ctx context.Context, exp *models.SdUser,
newVerificationCode string, newVerified bool) (*models.SdUser, error)
    GetByVerificationCode(ctx context.Context, verificationCode string)
(*models.SdUser, error)
    UpdatePasswordReset(ctx context.Context, exp *models.SdUser,
passwordResetToken string, passwordResetAt time.Time) (*models.SdUser, error)
    GetByResetToken(ctx context.Context, resetToken string) (*models.SdUser,
error)
    GetByResetTokenResetAt(ctx context.Context, resetToken string, resetAt
time.Time) (*models.SdUser, error)
    UpdatePasswordResetToken(ctx context.Context, exp *models.SdUser, newPassword
string, resetToken string) (*models.SdUser, error)
}
```

`redis_repository.go` **(Redis repository interface)**

```go
package users

import (
    "icmongolang/internal"
    "icmongolang/internal/models"
)

type UserRedisRepository interface {
    internal.RedisRepository[models.SdUser]
}
```

`usecase.go` **(UseCase interface)**

```go
package users

import (
    "context"

    "icmongolang/internal"
    "icmongolang/internal/models"

    "github.com/google/uuid"
)

type UserUseCaseI interface {
    internal.UseCaseI[models.SdUser]
    CreateUser(ctx context.Context, exp *models.SdUser, confirmPassword string)
(*models.SdUser, error)
    SignIn(ctx context.Context, email string, password string) (string, string,
error)
    IsActive(ctx context.Context, exp models.SdUser) bool
    IsSuper(ctx context.Context, exp models.SdUser) bool
    CreateSuperUserIfNotExist(context.Context) (bool, error)
    UpdatePassword(ctx context.Context, id uuid.UUID, oldPassword string,
newPassword string, confirmPassword string) (*models.SdUser, error)
    ParseIdFromRefreshToken(ctx context.Context, refreshToken string) (uuid.UUID,
error)
    Refresh(ctx context.Context, refreshToken string) (string, string, error)
    GenerateRedisUserKey(id uuid.UUID) string
    GenerateRedisRefreshTokenKey(id uuid.UUID) string
    Logout(ctx context.Context, refreshToken string) error
    LogoutAll(ctx context.Context, id uuid.UUID) error
    Verify(ctx context.Context, verificationCode string) error
    ForgotPassword(ctx context.Context, email string) error
    ResetPassword(ctx context.Context, resetToken string, newPassword string,
confirmPassword string) error
}
```

`worker.go` **(Async task definitions)**

```go
package users

import (
    "context"

    "github.com/hibiken/asynq"
)

const TaskSendEmail = "task:send_email"

type PayloadSendEmail struct {
    From      string `json:"from"`
    To        string `json:"to"`
```

```
    Subject   string `json:"subject"`
    BodyHtml  string `json:"bodyHtml"`
    BodyPlain string `json:"bodyPlain"`
}

type UserRedisTaskDistributor interface {
    DistributeTaskSendEmail(ctx context.Context, payload *PayloadSendEmail, opts
...asynq.Option) error
}

type UserRedisTaskProcessor interface {
    ProcessTaskSendEmail(ctx context.Context, task *asynq.Task) error
}
```

3.3 Delivery (HTTP)

`delivery/http/handlers.go`

```
package http

import (
    "encoding/json"
    "errors"
    "net/http"
    "strconv"
    "strings"

    "icmongolang/config"
    "icmongolang/internal/middleware"
    "icmongolang/internal/models"
    "icmongolang/internal/users"
    "icmongolang/internal/users/presenter"
    "icmongolang/pkg/httpErrors"
    "icmongolang/pkg/logger"
    "icmongolang/pkg/responses"
    "icmongolang/pkg/utils"

    "github.com/go-chi/chi/v5"
    "github.com/go-chi/render"
    "github.com/google/uuid"
)

type userHandler struct {
    cfg     *config.Config
    usersUC users.UserUseCaseI
    logger  logger.Logger
}

func CreateUserHandler(uc users.UserUseCaseI, cfg *config.Config, logger
logger.Logger) users.Handlers {
    return &userHandler{cfg: cfg, usersUC: uc, logger: logger}
```

```go
}

// Register - POST /register (public)
func (h *userHandler) Register() http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        req := new(presenter.UserCreate)
        if err := json.NewDecoder(r.Body).Decode(req); err != nil {
            render.Render(w, r, responses.CreateErrorResponse(err))
            return
        }
        if err := utils.ValidateStruct(r.Context(), req); err != nil {
            render.Render(w, r,
responses.CreateErrorResponse(httpErrors.ErrValidation(err)))
            return
        }
        newUser, err := h.usersUC.CreateUser(r.Context(), mapModel(req),
req.ConfirmPassword)
        if err != nil {
            render.Render(w, r, responses.CreateErrorResponse(err))
            return
        }
        userResp := mapModelResponse(newUser)
        if userResp == nil {
            render.Render(w, r, responses.CreateErrorResponse(errors.New("internal
server error")))
            return
        }
        render.Respond(w, r, responses.CreateSuccessResponse(*userResp))
    }
}

// Create - POST /user (admin only)
func (h *userHandler) Create() http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        req := new(presenter.UserCreate)
        if err := json.NewDecoder(r.Body).Decode(req); err != nil {
            render.Render(w, r, responses.CreateErrorResponse(err))
            return
        }
        if err := utils.ValidateStruct(r.Context(), req); err != nil {
            render.Render(w, r,
responses.CreateErrorResponse(httpErrors.ErrValidation(err)))
            return
        }
        newUser, err := h.usersUC.CreateUser(r.Context(), mapModel(req),
req.ConfirmPassword)
        if err != nil {
            render.Render(w, r, responses.CreateErrorResponse(err))
            return
        }
        render.Respond(w, r,
responses.CreateSuccessResponse(mapModelResponse(newUser)))
    }
}
```

```go
// Get - GET /user/{id}
func (h *userHandler) Get() http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        id, err := uuid.Parse(chi.URLParam(r, "id"))
        if err != nil {
            render.Render(w, r,
responses.CreateErrorResponse(httpErrors.ErrValidation(err)))
            return
        }
        user, err := h.usersUC.Get(r.Context(), id)
        if err != nil {
            render.Render(w, r, responses.CreateErrorResponse(err))
            return
        }
        render.Respond(w, r,
responses.CreateSuccessResponse(mapModelResponse(user)))
    }
}

// GetMulti - GET /user?limit=10&offset=0
func (h *userHandler) GetMulti() http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        q := r.URL.Query()
        limit, _ := strconv.Atoi(q.Get("limit"))
        offset, _ := strconv.Atoi(q.Get("offset"))
        users, err := h.usersUC.GetMulti(r.Context(), limit, offset)
        if err != nil {
            render.Render(w, r, responses.CreateErrorResponse(err))
            return
        }
        render.Respond(w, r,
responses.CreateSuccessResponse(mapModelsResponse(users)))
    }
}

// Delete - DELETE /user/{id} (admin only)
func (h *userHandler) Delete() http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        id, err := uuid.Parse(chi.URLParam(r, "id"))
        if err != nil {
            render.Render(w, r,
responses.CreateErrorResponse(httpErrors.ErrValidation(err)))
            return
        }
        user, err := h.usersUC.Delete(r.Context(), id)
        if err != nil {
            render.Render(w, r, responses.CreateErrorResponse(err))
            return
        }
        render.Respond(w, r,
responses.CreateSuccessResponse(mapModelResponse(user)))
    }
}
```

```go
// Update - PUT /user/{id} (admin only)
func (h *userHandler) Update() http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        id, err := uuid.Parse(chi.URLParam(r, "id"))
        if err != nil {
            render.Render(w, r,
responses.CreateErrorResponse(httpErrors.ErrValidation(err)))
            return
        }
        req := new(presenter.UserUpdate)
        if err := json.NewDecoder(r.Body).Decode(req); err != nil {
            render.Render(w, r, responses.CreateErrorResponse(err))
            return
        }
        values := make(map[string]interface{})
        if req.Firstname != nil {
            values["firstname"] = *req.Firstname
        }
        if req.Lastname != nil {
            values["lastname"] = *req.Lastname
        }
        if req.Fullname != nil {
            values["fullname"] = *req.Fullname
        }
        if req.MobileNumber != nil {
            values["mobile_number"] = *req.MobileNumber
        }
        if req.PhoneNumber != nil {
            values["phone_number"] = *req.PhoneNumber
        }
        if req.LineID != nil {
            values["lineid"] = *req.LineID
        }
        if req.LocationID != nil {
            values["location_id"] = *req.LocationID
        }
        updatedUser, err := h.usersUC.Update(r.Context(), id, values)
        if err != nil {
            render.Render(w, r, responses.CreateErrorResponse(err))
            return
        }
        render.Respond(w, r,
responses.CreateSuccessResponse(mapModelResponse(updatedUser)))
    }
}

// UpdatePassword - PATCH /user/{id}/updatepass (admin only)
func (h *userHandler) UpdatePassword() http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        id, err := uuid.Parse(chi.URLParam(r, "id"))
        if err != nil {
            render.Render(w, r,
responses.CreateErrorResponse(httpErrors.ErrValidation(err)))
```

```go
            return
        }
        req := new(presenter.UserUpdatePassword)
        if err := json.NewDecoder(r.Body).Decode(req); err != nil {
            render.Render(w, r, responses.CreateErrorResponse(err))
            return
        }
        if err := utils.ValidateStruct(r.Context(), req); err != nil {
            render.Render(w, r,
responses.CreateErrorResponse(httpErrors.ErrValidation(err)))
            return
        }
        updatedUser, err := h.usersUC.UpdatePassword(r.Context(), id,
req.OldPassword, req.NewPassword, req.ConfirmPassword)
        if err != nil {
            render.Render(w, r, responses.CreateErrorResponse(err))
            return
        }
        render.Respond(w, r,
responses.CreateSuccessResponse(mapModelResponse(updatedUser)))
    }
}

// UpdateRole - PATCH /user/{id}/role (admin only)
func (h *userHandler) UpdateRole() http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        id, err := uuid.Parse(chi.URLParam(r, "id"))
        if err != nil {
            render.Render(w, r,
responses.CreateErrorResponse(httpErrors.ErrValidation(err)))
            return
        }
        req := new(presenter.UserUpdateRole)
        if err := json.NewDecoder(r.Body).Decode(req); err != nil {
            render.Render(w, r, responses.CreateErrorResponse(err))
            return
        }
        if req.RoleID <= 0 {
            render.Render(w, r,
responses.CreateErrorResponse(httpErrors.ErrValidation(errors.New("invalid role
id"))))
            return
        }
        updatedUser, err := h.usersUC.Update(r.Context(), id,
map[string]interface{}{"role_id": req.RoleID})
        if err != nil {
            render.Render(w, r, responses.CreateErrorResponse(err))
            return
        }
        render.Respond(w, r,
responses.CreateSuccessResponse(mapModelResponse(updatedUser)))
    }
}
```

```go
// Me - GET /user/me
func (h *userHandler) Me() http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        user, err := middleware.GetUserFromCtx(r.Context())
        if err != nil {
            render.Render(w, r, responses.CreateErrorResponse(err))
            return
        }
        render.Respond(w, r,
responses.CreateSuccessResponse(mapModelResponse(user)))
    }
}

// UpdateMe - PUT /user/me
func (h *userHandler) UpdateMe() http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        user, err := middleware.GetUserFromCtx(r.Context())
        if err != nil {
            render.Render(w, r, responses.CreateErrorResponse(err))
            return
        }
        req := new(presenter.UserUpdate)
        if err := json.NewDecoder(r.Body).Decode(req); err != nil {
            render.Render(w, r, responses.CreateErrorResponse(err))
            return
        }
        values := make(map[string]interface{})
        if req.Firstname != nil {
            values["firstname"] = *req.Firstname
        }
        if req.Lastname != nil {
            values["lastname"] = *req.Lastname
        }
        if req.Fullname != nil {
            values["fullname"] = *req.Fullname
        }
        if req.MobileNumber != nil {
            values["mobile_number"] = *req.MobileNumber
        }
        if req.PhoneNumber != nil {
            values["phone_number"] = *req.PhoneNumber
        }
        if req.LineID != nil {
            values["lineid"] = *req.LineID
        }
        if req.LocationID != nil {
            values["location_id"] = *req.LocationID
        }
        updatedUser, err := h.usersUC.Update(r.Context(), user.ID, values)
        if err != nil {
            render.Render(w, r, responses.CreateErrorResponse(err))
            return
        }
        render.Respond(w, r,
```

```
responses.CreateSuccessResponse(mapModelResponse(updatedUser)))
    }
}

// UpdatePasswordMe - PATCH /user/me/updatepass
func (h *userHandler) UpdatePasswordMe() http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        user, err := middleware.GetUserFromCtx(r.Context())
        if err != nil {
            render.Render(w, r, responses.CreateErrorResponse(err))
            return
        }
        req := new(presenter.UserUpdatePassword)
        if err := json.NewDecoder(r.Body).Decode(req); err != nil {
            render.Render(w, r, responses.CreateErrorResponse(err))
            return
        }
        if err := utils.ValidateStruct(r.Context(), req); err != nil {
            render.Render(w, r,
responses.CreateErrorResponse(httpErrors.ErrValidation(err)))
            return
        }
        updatedUser, err := h.usersUC.UpdatePassword(r.Context(), user.ID,
req.OldPassword, req.NewPassword, req.ConfirmPassword)
        if err != nil {
            render.Render(w, r, responses.CreateErrorResponse(err))
            return
        }
        render.Respond(w, r,
responses.CreateSuccessResponse(mapModelResponse(updatedUser)))
    }
}

// LogoutAllAdmin - GET /user/{id}/logoutall (admin only)
func (h *userHandler) LogoutAllAdmin() http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        id, err := uuid.Parse(chi.URLParam(r, "id"))
        if err != nil {
            render.Render(w, r,
responses.CreateErrorResponse(httpErrors.ErrValidation(err)))
            return
        }
        if err := h.usersUC.LogoutAll(r.Context(), id); err != nil {
            render.Render(w, r, responses.CreateErrorResponse(err))
            return
        }
        render.Respond(w, r, responses.CreateSuccessResponse(struct{}{}))
    }
}

// ------------------------------
// Mapping functions
// ------------------------------
```

```go
func mapModel(req *presenter.UserCreate) *models.SdUser {
    var fullname *string
    if req.Fullname != "" {
        fullname = &req.Fullname
    } else if req.Firstname != "" || req.Lastname != "" {
        combined := strings.TrimSpace(req.Firstname + " " + req.Lastname)
        if combined != "" {
            fullname = &combined
        }
    }
    return &models.SdUser{
        Email:        strings.ToLower(strings.TrimSpace(req.Email)),
        Username:     strings.ToLower(strings.TrimSpace(req.Email)),
        Password:     req.Password,
        RoleID:       req.RoleID,
        Firstname:    stringPtr(req.Firstname),
        Lastname:     stringPtr(req.Lastname),
        Fullname:     fullname,
        MobileNumber: stringPtr(req.MobileNumber),
        PhoneNumber:  stringPtr(req.PhoneNumber),
        LineID:       stringPtr(req.LineID),
        LocationID:   stringPtr(req.LocationID),
        Status:       1,
        IsSuperUser:  false,
        Verified:     false,
    }
}

func mapModelResponse(user *models.SdUser) *presenter.UserResponse {
    if user == nil {
        return nil
    }
    return &presenter.UserResponse{
        ID:           user.ID,
        Email:        user.Email,
        RoleID:       user.RoleID,
        Firstname:    user.Firstname,
        Lastname:     user.Lastname,
        Fullname:     user.Fullname,
        MobileNumber: user.MobileNumber,
        PhoneNumber:  user.PhoneNumber,
        LineID:       user.LineID,
        LocationID:   user.LocationID,
        Status:       user.Status,
        IsSuperUser:  user.IsSuperUser,
        Verified:     user.Verified,
        CreatedAt:    user.CreatedDate,
        UpdatedAt:    user.UpdatedDate,
    }
}

func mapModelsResponse(users []*models.SdUser) []*presenter.UserResponse {
    out := make([]*presenter.UserResponse, len(users))
    for i, u := range users {
```

```go
        out[i] = mapModelResponse(u)
    }
    return out
}

func stringPtr(s string) *string {
    if s == "" {
        return nil
    }
    return &s
}
```

delivery/http/routes.go

```go
package http

import (
    "icmongolang/internal/middleware"
    "icmongolang/internal/users"

    "github.com/go-chi/chi/v5"
)

func MapUserRoute(router *chi.Mux, h users.Handlers, mw
*middleware.MiddlewareManager) {
    // Public route (no auth)
    router.Post("/register", h.Register())

    // User routes (all require auth)
    router.Route("/user", func(r chi.Router) {
        r.Group(func(r chi.Router) {
            r.Use(mw.Verifier(true))
            r.Use(mw.Authenticator())
            r.Use(mw.CurrentUser())
            r.Use(mw.ActiveUser())

            r.Get("/me", h.Me())
            r.Put("/me", h.UpdateMe())
            r.Patch("/me/updatepass", h.UpdatePasswordMe())

            // Admin only
            r.Group(func(r chi.Router) {
                r.Use(mw.SuperUser())
                r.Get("/", h.GetMulti())
                r.Post("/", h.Create())
                r.Patch("/{id}/role", h.UpdateRole())
            })

            // Per-id routes (admin for write)
            r.Route("/{id}", func(r chi.Router) {
                r.Get("/", h.Get())
```

```
                r.Group(func(r chi.Router) {
                    r.Use(mw.SuperUser())
                    r.Delete("/", h.Delete())
                    r.Put("/", h.Update())
                    r.Patch("/updatepass", h.UpdatePassword())
                    r.Get("/logoutall", h.LogoutAllAdmin())
                })
            })
        })
    })
}
```

3.4 Usecase

`usecase/usecase.go` **(implementation)**

```
package usecase

import (
    "context"
    "errors"
    "fmt"
    "strings"
    "time"

    "icmongolang/config"
    "icmongolang/internal/models"
    "icmongolang/internal/usecase"
    "icmongolang/internal/users"
    "icmongolang/internal/worker"
    "icmongolang/pkg/cryptpass"
    "icmongolang/pkg/emailTemplates"
    "icmongolang/pkg/httpErrors"
    "icmongolang/pkg/jwt"
    "icmongolang/pkg/logger"
    "icmongolang/pkg/secureRandom"

    "github.com/google/uuid"
    "github.com/hibiken/asynq"
)

type userUseCase struct {
    usecase.UseCase[models.SdUser]
    pgRepo                 users.UserPgRepository
    redisRepo              users.UserRedisRepository
    emailTemplateGenerator emailTemplates.EmailTemplatesGenerator
    redisTaskDistributor   users.UserRedisTaskDistributor
}

func CreateUserUseCaseI(
    pgRepo users.UserPgRepository,
```

```go
    redisRepo users.UserRedisRepository,
    redisTaskDistributor users.UserRedisTaskDistributor,
    cfg *config.Config,
    logger logger.Logger,
) users.UserUseCaseI {
    return &userUseCase{
        UseCase:                usecase.CreateUseCase[models.SdUser](pgRepo, cfg,
logger),
        pgRepo:                 pgRepo,
        redisRepo:              redisRepo,
        emailTemplateGenerator: emailTemplates.NewEmailTemplatesGenerator(cfg),
        redisTaskDistributor:   redisTaskDistributor,
    }
}

// Get with Redis cache
func (u *userUseCase) Get(ctx context.Context, id uuid.UUID) (*models.SdUser,
error) {
    cachedUser, err := u.redisRepo.Get(ctx, u.GenerateRedisUserKey(id))
    if err != nil {
        return nil, err
    }
    if cachedUser != nil {
        return cachedUser, nil
    }
    user, err := u.pgRepo.Get(ctx, id)
    if err != nil {
        return nil, err
    }
    if err = u.redisRepo.Create(ctx, u.GenerateRedisUserKey(id), user, 3600); err
!= nil {
        return nil, err
    }
    return user, nil
}

// Delete and invalidate cache
func (u *userUseCase) Delete(ctx context.Context, id uuid.UUID) (*models.SdUser,
error) {
    user, err := u.pgRepo.Delete(ctx, id)
    if err != nil {
        return nil, err
    }
    _ = u.redisRepo.Delete(ctx, u.GenerateRedisUserKey(id))
    _ = u.redisRepo.Delete(ctx, u.GenerateRedisRefreshTokenKey(id))
    return user, nil
}

// Update and invalidate cache
func (u *userUseCase) Update(ctx context.Context, id uuid.UUID, values
map[string]interface{}) (*models.SdUser, error) {
    obj, err := u.Get(ctx, id)
    if err != nil || obj == nil {
        return nil, err
```

```
    }
    user, err := u.pgRepo.Update(ctx, obj, values)
    if err != nil {
        return nil, err
    }
    _ = u.redisRepo.Delete(ctx, u.GenerateRedisUserKey(id))
    return user, nil
}

// Create – inserts user and sends verification email
func (u *userUseCase) Create(ctx context.Context, exp *models.SdUser)
(*models.SdUser, error) {
    exp.Email = strings.ToLower(strings.TrimSpace(exp.Email))
    exp.Password = strings.TrimSpace(exp.Password)

    hashedPassword, err := cryptpass.HashPassword(exp.Password)
    if err != nil {
        return nil, err
    }
    exp.Password = hashedPassword
    if exp.Username == "" {
        exp.Username = exp.Email
    }
    if exp.RoleID == 0 {
        exp.RoleID = 2
    }

    user, err := u.pgRepo.Create(ctx, exp)
    if err != nil {
        return nil, err
    }
    if user.Verified {
        return user, nil
    }

    verificationCode, err := secureRandom.RandomHex(16)
    if err != nil {
        return nil, err
    }
    updatedUser, err := u.pgRepo.UpdateVerificationCode(ctx, user,
verificationCode)
    if err != nil {
        return nil, err
    }

    name := ""
    if updatedUser.Fullname != nil {
        name = *updatedUser.Fullname
    } else {
        name = updatedUser.Email
    }
    bodyHtml, bodyPlain, err :=
u.emailTemplateGenerator.GenerateVerificationCodeTemplate(
        ctx,
```

```
        name,
        fmt.Sprintf("http://localhost:5000/auth/verifyemail?code=%s",
verificationCode),
    )
    if err != nil {
        return nil, err
    }

    err = u.redisTaskDistributor.DistributeTaskSendEmail(ctx,
&users.PayloadSendEmail{
        From:      u.Cfg.Email.From,
        To:        updatedUser.Email,
        Subject:   u.Cfg.Email.VerificationSubject,
        BodyHtml:  bodyHtml,
        BodyPlain: bodyPlain,
    }, asynq.MaxRetry(10), asynq.ProcessIn(10*time.Second),
asynq.Queue(worker.QueueCritical))
    if err != nil {
        return nil, err
    }
    return updatedUser, nil
}

// CreateUser with password confirmation
func (u *userUseCase) CreateUser(ctx context.Context, exp *models.SdUser,
confirmPassword string) (*models.SdUser, error) {
    if exp.Password != confirmPassword {
        return nil, httpErrors.ErrValidation(errors.New("password do not match"))
    }
    return u.Create(ctx, exp)
}

// createToken generates access & refresh tokens
func (u *userUseCase) createToken(ctx context.Context, exp models.SdUser) (string,
string, error) {
    accessToken, err := jwt.CreateAccessTokenRS256(
        exp.ID.String(),
        exp.Email,
        u.Cfg.Jwt.AccessTokenPrivateKey,
        u.Cfg.Jwt.AccessTokenExpireDuration*int64(time.Minute),
        u.Cfg.Jwt.Issuer,
    )
    if err != nil {
        return "", "", err
    }
    refreshToken, err := jwt.CreateAccessTokenRS256(
        exp.ID.String(),
        exp.Email,
        u.Cfg.Jwt.RefreshTokenPrivateKey,
        u.Cfg.Jwt.RefreshTokenExpireDuration*int64(time.Minute),
        u.Cfg.Jwt.Issuer,
    )
    if err != nil {
        return "", "", err
```

```go
    }
    return accessToken, refreshToken, nil
}

// SignIn authenticates user
func (u *userUseCase) SignIn(ctx context.Context, email string, password string)
(string, string, error) {
    user, err := u.pgRepo.GetByEmail(ctx, email)
    if err != nil {
        return "", "", httpErrors.ErrNotFound(err)
    }
    if !cryptpass.ComparePassword(password, user.Password) {
        return "", "", httpErrors.ErrWrongPassword(errors.New("wrong password"))
    }
    accessToken, refreshToken, err := u.createToken(ctx, *user)
    if err != nil {
        return "", "", err
    }
    if err = u.redisRepo.Sadd(ctx, u.GenerateRedisRefreshTokenKey(user.ID),
refreshToken); err != nil {
        return "", "", err
    }
    return accessToken, refreshToken, nil
}

// IsActive checks if user is active
func (u *userUseCase) IsActive(ctx context.Context, exp models.SdUser) bool {
    return exp.Status == 1
}

// IsSuper checks superuser
func (u *userUseCase) IsSuper(ctx context.Context, exp models.SdUser) bool {
    return exp.IsSuperUser
}

// CreateSuperUserIfNotExist from config
func (u *userUseCase) CreateSuperUserIfNotExist(ctx context.Context) (bool, error)
{
    user, err := u.pgRepo.GetByEmail(ctx, u.Cfg.FirstSuperUser.Email)
    if err != nil || user == nil {
        fullname := u.Cfg.FirstSuperUser.Name
        _, err := u.Create(ctx, &models.SdUser{
            Fullname:    &fullname,
            Email:       u.Cfg.FirstSuperUser.Email,
            Username:    u.Cfg.FirstSuperUser.Email,
            Password:    u.Cfg.FirstSuperUser.Password,
            RoleID:      1,
            Status:      1,
            IsSuperUser: true,
            Verified:    true,
        })
        if err != nil {
            return false, err
        }
```

```go
        return true, nil
    }
    return false, nil
}

// UpdatePassword changes password and invalidates all sessions
func (u *userUseCase) UpdatePassword(ctx context.Context, id uuid.UUID,
oldPassword, newPassword, confirmPassword string) (*models.SdUser, error) {
    if newPassword != confirmPassword {
        return nil, httpErrors.ErrValidation(errors.New("password do not match"))
    }
    user, err := u.Get(ctx, id)
    if err != nil {
        return nil, err
    }
    if !cryptpass.ComparePassword(oldPassword, user.Password) {
        return nil, httpErrors.ErrWrongPassword(errors.New("old password and new
password not same"))
    }
    hashedPassword, err := cryptpass.HashPassword(newPassword)
    if err != nil {
        return nil, err
    }
    updatedUser, err := u.pgRepo.UpdatePassword(ctx, user, hashedPassword)
    if err != nil {
        return nil, err
    }
    _ = u.redisRepo.Delete(ctx, u.GenerateRedisUserKey(id))
    _ = u.redisRepo.Delete(ctx, u.GenerateRedisRefreshTokenKey(id))
    return updatedUser, nil
}

// ParseIdFromRefreshToken extracts user ID from refresh token
func (u *userUseCase) ParseIdFromRefreshToken(ctx context.Context, refreshToken
string) (uuid.UUID, error) {
    id, _, err := jwt.ParseTokenRS256(refreshToken,
u.Cfg.Jwt.RefreshTokenPublicKey)
    if err != nil {
        return uuid.UUID{}, err
    }
    idParsed, err := uuid.Parse(id)
    if err != nil {
        return uuid.UUID{}, httpErrors.ErrInvalidJWTClaims(errors.New("can not
convert id to uuid from id in token"))
    }
    return idParsed, nil
}

// Refresh issues new tokens using a valid refresh token
func (u *userUseCase) Refresh(ctx context.Context, refreshToken string) (string,
string, error) {
    idParsed, err := u.ParseIdFromRefreshToken(ctx, refreshToken)
    if err != nil {
        return "", "", err
```

```go
    }
    isMember, err := u.redisRepo.SIsMember(ctx,
u.GenerateRedisRefreshTokenKey(idParsed), refreshToken)
    if err != nil {
        return "", "", err
    }
    if !isMember {
        return "", "", httpErrors.ErrNotFoundRefreshTokenRedis(errors.New("not
found refresh token in redis"))
    }
    if err = u.redisRepo.Srem(ctx, u.GenerateRedisRefreshTokenKey(idParsed),
refreshToken); err != nil {
        return "", "", err
    }
    user, err := u.Get(ctx, idParsed)
    if err != nil {
        return "", "", err
    }
    accessToken, newRefreshToken, err := u.createToken(ctx, *user)
    if err != nil {
        return "", "", err
    }
    if err = u.redisRepo.Sadd(ctx, u.GenerateRedisRefreshTokenKey(user.ID),
newRefreshToken); err != nil {
        return "", "", err
    }
    return accessToken, newRefreshToken, nil
}

// Logout removes specific refresh token
func (u *userUseCase) Logout(ctx context.Context, refreshToken string) error {
    idParsed, err := u.ParseIdFromRefreshToken(ctx, refreshToken)
    if err != nil {
        return err
    }
    return u.redisRepo.Srem(ctx, u.GenerateRedisRefreshTokenKey(idParsed),
refreshToken)
}

// LogoutAll removes all refresh tokens of a user
func (u *userUseCase) LogoutAll(ctx context.Context, id uuid.UUID) error {
    return u.redisRepo.Delete(ctx, u.GenerateRedisRefreshTokenKey(id))
}

// Verify confirms email
func (u *userUseCase) Verify(ctx context.Context, verificationCode string) error {
    user, err := u.pgRepo.GetByVerificationCode(ctx, verificationCode)
    if err != nil {
        return err
    }
    if user.Verified {
        return httpErrors.ErrUserAlreadyVerified(errors.New("user already
verified"))
    }
```

```go
    updatedUser, err := u.pgRepo.UpdateVerification(ctx, user, "", true)
    if err != nil {
        return err
    }
    _ = u.redisRepo.Delete(ctx, u.GenerateRedisUserKey(updatedUser.ID))
    return nil
}

// ForgotPassword sends reset email
func (u *userUseCase) ForgotPassword(ctx context.Context, email string) error {
    user, err := u.pgRepo.GetByEmail(ctx, email)
    if err != nil {
        return httpErrors.ErrNotFound(err)
    }
    if !user.Verified {
        return httpErrors.ErrUserNotVerified(errors.New("user not verified"))
    }
    resetToken, err := secureRandom.RandomHex(16)
    if err != nil {
        return err
    }
    updatedUser, err := u.pgRepo.UpdatePasswordReset(ctx, user, resetToken,
time.Now().Add(15*time.Minute))
    if err != nil {
        return err
    }
    _ = u.redisRepo.Delete(ctx, u.GenerateRedisUserKey(updatedUser.ID))

    name := ""
    if updatedUser.Fullname != nil {
        name = *updatedUser.Fullname
    } else {
        name = updatedUser.Email
    }
    bodyHtml, bodyPlain, err :=
u.emailTemplateGenerator.GeneratePasswordResetTemplate(
        ctx,
        name,
        fmt.Sprintf("http://localhost:5000/auth/resetpassword?code=%s",
resetToken),
    )
    if err != nil {
        return err
    }
    return u.redisTaskDistributor.DistributeTaskSendEmail(ctx,
&users.PayloadSendEmail{
        From:      u.Cfg.Email.From,
        To:        updatedUser.Email,
        Subject:   u.Cfg.Email.ResetSubject,
        BodyHtml:  bodyHtml,
        BodyPlain: bodyPlain,
    }, asynq.MaxRetry(10), asynq.ProcessIn(10*time.Second),
asynq.Queue(worker.QueueCritical))
}
```

```go
// ResetPassword performs password reset
func (u *userUseCase) ResetPassword(ctx context.Context, resetToken, newPassword,
confirmPassword string) error {
    if newPassword != confirmPassword {
        return httpErrors.ErrValidation(errors.New("password do not match"))
    }
    user, err := u.pgRepo.GetByResetTokenResetAt(ctx, resetToken, time.Now())
    if err != nil {
        return err
    }
    hashedPassword, err := cryptpass.HashPassword(newPassword)
    if err != nil {
        return err
    }
    updatedUser, err := u.pgRepo.UpdatePasswordResetToken(ctx, user,
hashedPassword, "")
    if err != nil {
        return err
    }
    _ = u.redisRepo.Delete(ctx, u.GenerateRedisUserKey(updatedUser.ID))
    _ = u.redisRepo.Delete(ctx, u.GenerateRedisRefreshTokenKey(updatedUser.ID))
    return nil
}

// GenerateRedisUserKey returns cache key for user
func (u *userUseCase) GenerateRedisUserKey(id uuid.UUID) string {
    return fmt.Sprintf("%s:%s", models.SdUser{}.TableName(), id.String())
}

// GenerateRedisRefreshTokenKey returns key for refresh token set
func (u *userUseCase) GenerateRedisRefreshTokenKey(id uuid.UUID) string {
    return fmt.Sprintf("RefreshToken:%s", id.String())
}
```

3.5 Repository

repository/pg_repository.go

```go
package repository

import (
    "context"
    "time"

    "icmongolang/internal/models"
    "icmongolang/internal/repository"
    "icmongolang/internal/users"

    "gorm.io/gorm"
)
```

```go
type UserPgRepo struct {
    repository.PgRepo[models.SdUser]
}

func RegisterUserPgRepository(db *gorm.DB) users.UserPgRepository {
    return &UserPgRepo{
        PgRepo: repository.CreatePgRepo[models.SdUser](db),
    }
}

func CreateUserPgRepository(db *gorm.DB) users.UserPgRepository {
    return &UserPgRepo{
        PgRepo: repository.CreatePgRepo[models.SdUser](db),
    }
}

func (r *UserPgRepo) GetByEmail(ctx context.Context, email string)
(*models.SdUser, error) {
    var obj *models.SdUser
    if result := r.DB.WithContext(ctx).First(&obj, "email = ?", email);
result.Error != nil {
        return nil, result.Error
    }
    return obj, nil
}

func (r *UserPgRepo) UpdatePassword(ctx context.Context, exp *models.SdUser,
newPassword string) (*models.SdUser, error) {
    if result := r.DB.WithContext(ctx).Model(&exp).Select("password").
        Updates(map[string]interface{}{"password": newPassword}); result.Error !=
nil {
        return nil, result.Error
    }
    return exp, nil
}

func (r *UserPgRepo) UpdateVerificationCode(ctx context.Context, exp
*models.SdUser, newVerificationCode string) (*models.SdUser, error) {
    if result := r.DB.WithContext(ctx).Model(&exp).Select("verification_code").
        Updates(map[string]interface{}{"verification_code": newVerificationCode});
result.Error != nil {
        return nil, result.Error
    }
    return exp, nil
}

func (r *UserPgRepo) UpdateVerification(ctx context.Context, exp *models.SdUser,
newVerificationCode string, newVerified bool) (*models.SdUser, error) {
    if result := r.DB.WithContext(ctx).Model(&exp).Select("verification_code",
"verified").
        Updates(map[string]interface{}{
            "verification_code": newVerificationCode,
            "verified":          newVerified,
```

```go
        }); result.Error != nil {
        return nil, result.Error
    }
    return exp, nil
}

func (r *UserPgRepo) GetByVerificationCode(ctx context.Context, verificationCode
string) (*models.SdUser, error) {
    var obj *models.SdUser
    if result := r.DB.WithContext(ctx).First(&obj, "verification_code = ?",
verificationCode); result.Error != nil {
        return nil, result.Error
    }
    return obj, nil
}

func (r *UserPgRepo) UpdatePasswordReset(ctx context.Context, exp *models.SdUser,
passwordResetToken string, passwordResetAt time.Time) (*models.SdUser, error) {
    if result := r.DB.WithContext(ctx).Model(&exp).Select("password_reset_token",
"password_reset_at").
        Updates(map[string]interface{}{
            "password_reset_token": passwordResetToken,
            "password_reset_at":    passwordResetAt,
        }); result.Error != nil {
        return nil, result.Error
    }
    return exp, nil
}

func (r *UserPgRepo) GetByResetToken(ctx context.Context, resetToken string)
(*models.SdUser, error) {
    var obj *models.SdUser
    if result := r.DB.WithContext(ctx).First(&obj, "reset_token = ?", resetToken);
result.Error != nil {
        return nil, result.Error
    }
    return obj, nil
}

func (r *UserPgRepo) GetByResetTokenResetAt(ctx context.Context, resetToken
string, resetAt time.Time) (*models.SdUser, error) {
    var obj *models.SdUser
    if result := r.DB.WithContext(ctx).First(&obj, "password_reset_token = ? AND
password_reset_at > ?", resetToken, resetAt); result.Error != nil {
        return nil, result.Error
    }
    return obj, nil
}

func (r *UserPgRepo) UpdatePasswordResetToken(ctx context.Context, exp
*models.SdUser, newPassword string, resetToken string) (*models.SdUser, error) {
    if result := r.DB.WithContext(ctx).Model(&exp).Select("password",
"password_reset_token").
        Updates(map[string]interface{}{"password": newPassword,
```

```
"password_reset_token": resetToken}); result.Error != nil {
        return nil, result.Error
    }
    return exp, nil
}
```

repository/redis_repository.go

```
package repository

import (
    "icmongolang/internal/models"
    "icmongolang/internal/repository"
    "icmongolang/internal/users"

    "github.com/redis/go-redis/v9"
)

type UserRedisRepo struct {
    repository.RedisRepo[models.SdUser]
}

func CreateUserRedisRepository(redisClient *redis.Client)
users.UserRedisRepository {
    return &UserRedisRepo{
        RedisRepo: repository.CreateRedisRepo[models.SdUser](redisClient),
    }
}
```

3.6 Distributor & Processor (Async Tasks)

distributor/distributor.go

```
package distributor

import (
    "context"
    "encoding/json"
    "fmt"

    "icmongolang/config"
    "icmongolang/internal/distributor"
    "icmongolang/internal/users"
    "icmongolang/pkg/logger"

    "github.com/hibiken/asynq"
)
```

```go
type userRedisTaskDistributor struct {
    distributor.RedisTaskDistributor
}

func NewUserRedisTaskDistributor(redisClient *asynq.Client, cfg *config.Config,
logger logger.Logger) users.UserRedisTaskDistributor {
    return &userRedisTaskDistributor{
        RedisTaskDistributor: distributor.NewRedisTaskDistributor(redisClient,
cfg, logger),
    }
}

func (d *userRedisTaskDistributor) DistributeTaskSendEmail(ctx context.Context,
payload *users.PayloadSendEmail, opts ...asynq.Option) error {
    jsonPayload, err := json.Marshal(payload)
    if err != nil {
        return fmt.Errorf("failed to marshal task payload %w", err)
    }
    task := asynq.NewTask(users.TaskSendEmail, jsonPayload, opts...)
    info, err := d.RedisClient.EnqueueContext(ctx, task)
    if err != nil {
        return fmt.Errorf("failed to enqueue task: %w", err)
    }
    d.Logger.Infof("Type: %v, Queue: %v, Max-Retry: %v, Msg: queued task",
task.Type(), info.Queue, info.MaxRetry)
    return nil
}
```

processor/processor.go

```go
package processor

import (
    "context"
    "encoding/json"
    "fmt"

    "icmongolang/config"
    "icmongolang/internal/processor"
    "icmongolang/internal/users"
    "icmongolang/pkg/logger"
    "icmongolang/pkg/sendEmail"

    "github.com/hibiken/asynq"
)

type userRedisTaskProcessor struct {
    processor.RedisTaskProcessor
    emailSender sendEmail.EmailSender
}
```

```go
func NewUserRedisTaskProcessor(server *asynq.Server, cfg *config.Config, logger
logger.Logger, emailSender sendEmail.EmailSender) users.UserRedisTaskProcessor {
    return &userRedisTaskProcessor{
        RedisTaskProcessor: processor.NewRedisTaskProcessor(server, cfg, logger),
        emailSender:        emailSender,
    }
}

func (p *userRedisTaskProcessor) ProcessTaskSendEmail(ctx context.Context, task
*asynq.Task) error {
    var payload users.PayloadSendEmail
    if err := json.Unmarshal(task.Payload(), &payload); err != nil {
        return fmt.Errorf("failed to unmarshal payload: %w", asynq.SkipRetry)
    }
    if err := p.emailSender.SendEmail(ctx, payload.From, payload.To,
payload.Subject, payload.BodyHtml, payload.BodyPlain); err != nil {
        return fmt.Errorf("failed to send verify email: %w", err)
    }
    p.Logger.Infof("Type: %v, Msg: email sended", task.Type())
    return nil
}
```

3.7 Presenter (DTO)

presenter/presenters.go

```go
package presenter

import (
    "time"

    "github.com/google/uuid"
)

// Register - used when creating a new user
type Register struct {
    Email           string `json:"email" validate:"required,email"`
    Password        string `json:"password" validate:"required,min=8"`
    ConfirmPassword string `json:"confirm_password" validate:"required,min=8"`
    RoleID          int    `json:"role_id" validate:"required,min=1"`
    Firstname       string `json:"firstname,omitempty"`
    Lastname        string `json:"lastname,omitempty"`
    Fullname        string `json:"fullname,omitempty"`
    MobileNumber    string `json:"mobile_number,omitempty"`
    PhoneNumber     string `json:"phone_number,omitempty"`
    LineID          string `json:"line_id,omitempty"`
    LocationID      string `json:"location_id,omitempty"`
}

// UserCreate - used when creating a new user
type UserCreate struct {
```

```go
    Email           string `json:"email" validate:"required,email"`
    Password        string `json:"password" validate:"required,min=8"`
    ConfirmPassword string `json:"confirm_password" validate:"required,min=8"`
    RoleID          int    `json:"role_id" validate:"required,min=1"`
    Firstname       string `json:"firstname,omitempty"`
    Lastname        string `json:"lastname,omitempty"`
    Fullname        string `json:"fullname,omitempty"`
    MobileNumber    string `json:"mobile_number,omitempty"`
    PhoneNumber     string `json:"phone_number,omitempty"`
    LineID          string `json:"line_id,omitempty"`
    LocationID      string `json:"location_id,omitempty"`
}

// UserUpdate – all fields optional (pointer)
type UserUpdate struct {
    Firstname    *string `json:"firstname,omitempty"`
    Lastname     *string `json:"lastname,omitempty"`
    Fullname     *string `json:"fullname,omitempty"`
    MobileNumber *string `json:"mobile_number,omitempty"`
    PhoneNumber  *string `json:"phone_number,omitempty"`
    LineID       *string `json:"line_id,omitempty"`
    LocationID   *string `json:"location_id,omitempty"`
}

// UserUpdateRole – used by admin
type UserUpdateRole struct {
    RoleID int `json:"role_id" validate:"required"`
}

// UserUpdatePassword – used when changing password
type UserUpdatePassword struct {
    OldPassword     string `json:"old_password" validate:"required,min=8"`
    NewPassword     string `json:"new_password" validate:"required,min=8"`
    ConfirmPassword string `json:"confirm_password" validate:"required,min=8"`
}

// UserResponse – full user data
type UserResponse struct {
    ID           uuid.UUID `json:"id"`
    Email        string    `json:"email"`
    RoleID       int       `json:"role_id"`
    Firstname    *string   `json:"firstname,omitempty"`
    Lastname     *string   `json:"lastname,omitempty"`
    Fullname     *string   `json:"fullname,omitempty"`
    MobileNumber *string   `json:"mobile_number,omitempty"`
    PhoneNumber  *string   `json:"phone_number,omitempty"`
    LineID       *string   `json:"line_id,omitempty"`
    LocationID   *string   `json:"location_id,omitempty"`
    Status       int16     `json:"status"`
    IsSuperUser  bool      `json:"is_superuser"`
    Verified     bool      `json:"verified"`
    CreatedAt    time.Time `json:"created_at"`
    UpdatedAt    time.Time `json:"updated_at"`
}
```

```go
// Auth related DTOs
type UserSignIn struct {
    Email    string `json:"email" validate:"required"`
    Password string `json:"password" validate:"required,min=8"`
}

type Token struct {
    AccessToken  string `json:"access_token,omitempty"`
    RefreshToken string `json:"refresh_token,omitempty"`
    TokenType    string `json:"token_type,omitempty"`
}

type PublicKey struct {
    PublicKeyAccessToken  string `json:"public_key_access_token,omitempty"`
    PublicKeyRefreshToken string `json:"public_key_refresh_token,omitempty"`
}

type ForgotPassword struct {
    Email string `json:"email" validate:"required"`
}

type ResetPassword struct {
    NewPassword     string `json:"new_password" validate:"required,min=8"`
    ConfirmPassword string `json:"confirm_password" validate:"required,min=8"`
}
```

---

4. วิธีการเพิ่มฟังก์ชันหรือแก้ไข (Extension Guide)

4.1 ขั้นตอนทั่วไปในการเพิ่ม endpoint ใหม่

สมมติต้องการเพิ่ม endpoint `POST /user/change-email` (ต้อง login)

ขั้นตอน:

1. **เพิ่ม DTO** ใน `presenter/presenters.go`

```go
type ChangeEmailRequest struct {
    NewEmail string `json:"new_email" validate:"required,email"`
}
```

2. **เพิ่ม interface method** ใน `usecase.go` (interface)

```go
ChangeEmail(ctx context.Context, userID uuid.UUID, newEmail string) error
```

3. **implement ใน** `usecase/usecase.go`

```go
func (u *userUseCase) ChangeEmail(ctx context.Context, userID uuid.UUID,
newEmail string) error {
    // validation, check duplicate, update DB, invalidate cache, etc.
}
```

4. **เพิ่ม handler method** ใน `delivery/http/handlers.go`

```go
func (h *userHandler) ChangeEmail() http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        user, _ := middleware.GetUserFromCtx(r.Context())
        req := new(presenter.ChangeEmailRequest)
        // decode, validate, call usecase, return response
    }
}
```

5. **เพิ่ม route** ใน `delivery/http/routes.go` ภายใต้ group ที่ต้องการ

```go
r.Post("/change-email", h.ChangeEmail())
```

6. **เพิ่ม interface method** ใน `handler.go` (ถ้าเป็น public interface)

```go
ChangeEmail() http.HandlerFunc
```

4.2 การเพิ่ม OTP (One-Time Password) โดยไม่เปลี่ยนโครงสร้างเดิม

เนื่องจากไม่ต้องการแก้ `models.SdUser` ให้ใช้ Redis จัดเก็บ OTP ชั่วคราว

ขั้นตอน:

1. **เพิ่ม DTO** ใน `presenter/presenters.go`

```go
type RequestOTPRequest struct {
    Email   string `json:"email" validate:"required,email"`
    Purpose string `json:"purpose"` // "verify", "2fa", etc.
}
type VerifyOTPRequest struct {
    Email string `json:"email" validate:"required,email"`
    OTP   string `json:"otp" validate:"required,len=6"`
}
```

2. **เพิ่ม interface methods** ใน `usecase.go`

```
RequestOTP(ctx context.Context, email string, purpose string) error
VerifyOTP(ctx context.Context, email string, otp string) error
```

3. **Implement ใน** `usecase/usecase.go`

```go
const otpTTL = 5 * time.Minute

func (u *userUseCase) RequestOTP(ctx context.Context, email string, purpose
string) error {
    // 1. check user exists
    user, err := u.pgRepo.GetByEmail(ctx, email)
    if err != nil {
        return httpErrors.ErrNotFound(err)
    }
    // 2. generate random 6-digit OTP
    otp := fmt.Sprintf("%06d", rand.Intn(1000000))
    // 3. store in Redis with TTL
    key := fmt.Sprintf("otp:%s:%s", email, purpose)
    if err := u.redisRepo.Set(ctx, key, otp, otpTTL); err != nil {
        return err
    }
    // 4. send OTP via email (async)
    // reuse existing task distributor with custom template
    return nil
}

func (u *userUseCase) VerifyOTP(ctx context.Context, email string, otp
string) error {
    // we need to know purpose, maybe from request
    // for simplicity, try common purposes
    purposes := []string{"verify", "2fa", "reset"}
    for _, p := range purposes {
        key := fmt.Sprintf("otp:%s:%s", email, p)
        stored, err := u.redisRepo.Get(ctx, key)
        if err == nil && stored == otp {
            // success: delete OTP
            _ = u.redisRepo.Delete(ctx, key)
            // optionally mark user as phone_verified (if you add a field
later)
            return nil
        }
    }
    return httpErrors.ErrInvalidOTP(errors.New("invalid or expired OTP"))
}
```

4. **เพิ่ม handlers** และ **routes** (public group)

4.3 การเพิ่ม 2FA (TOTP) แบบแยกตาราง

เนื่องจากไม่ต้องการเปลี่ยน `sd_user` ให้สร้างตารางใหม่ `user_totp`

```
CREATE TABLE user_totp (
    user_id UUID PRIMARY KEY REFERENCES sd_user(id),
    secret TEXT NOT NULL,
    enabled BOOLEAN DEFAULT false,
    created_at TIMESTAMP DEFAULT NOW()
);
```

สร้าง repository ใหม่ และ inject เข้า usecase

4.4 การเพิ่ม Social Login (Google/Facebook)

- เพิ่มตาราง `user_oauth` (provider, provider_id, user_id)
- เพิ่ม endpoint `/auth/google`, `/auth/google/callback`
- เพิ่ม usecase method `CreateOrGetUserFromOAuth`

4.5 การเพิ่ม Rate Limiting

ใช้ `go-chi/httprate`:

```
import "github.com/go-chi/httprate"

router.Group(func(r chi.Router) {
    r.Use(httprate.LimitByIP(5, 1*time.Minute))
    r.Post("/register", h.Register())
    r.Post("/forgot-password", h.ForgotPassword())
    r.Post("/request-otp", h.RequestOTP())
})
```

4.6 การเปลี่ยนจาก Asynq ไปใช้ระบบอื่น (RabbitMQ, Kafka)

ต้อง implement interface `UserRedisTaskDistributor` และ `UserRedisTaskProcessor` ใหม่ โดยเปลี่ยน logic ภายใน method `DistributeTaskSendEmail` และ `ProcessTaskSendEmail` โดยไม่ต้องแก้ usecase (Dependency Inversion)

---

5. Checklist ทดสอบ Module

Functional Tests

- ☐ Register สำเร็จ, ได้รับอีเมล verification
- ☐ Verify email ด้วย code → verified=true
- ☐ Sign in ด้วย email/password ถูกต้อง ได้ access + refresh token
- ☐ Sign in ด้วย password ผิด → error
- ☐ Access token หมดอายุ → 401
- ☐ Refresh token ใช้แล้วได้ token คู่ใหม่

- ☐ Logout → refresh token ถูกลบจาก Redis
- ☐ LogoutAll → ลบทุก refresh token
- ☐ Change password → ต้องใช้ old password, หลังเปลี่ยน refresh token เดิมใช้ไม่ได้
- ☐ Forgot password → ส่ง reset link ทางอีเมล
- ☐ Reset password → token ถูกต้อง, เปลี่ยนรหัสผ่านได้
- ☐ Admin: Create user, GetMulti, Update role, Delete
- ☐ Me: ดึงข้อมูลตัวเอง, UpdateMe, UpdatePasswordMe

Non-Functional

- ☐ Redis cache ทำงาน (GET user ครั้งแรกไป DB, ครั้งต่อไป cache)
- ☐ Cache invalidation เมื่อ update/delete
- ☐ Async email ถูก enqueue และ worker ส่งได้จริง
- ☐ Concurrent login: user login หลาย device → refresh tokens หลายตัวใน Redis set

Security

- ☐ Password ถูก hash (bcrypt)
- ☐ JWT signed ด้วย RS256
- ☐ Refresh token reuse detection (optional แต่แนะนำ)
- ☐ ไม่มี sensitive data ใน response

OTP (ถ้าเพิ่ม)

- ☐ Request OTP → ได้ OTP ใน Redis
- ☐ Verify OTP ถูกต้อง → สำเร็จ
- ☐ Verify OTP ผิด/หมดอายุ → error
- ☐ Rate limiting ป้องกัน brute force

เอกสารนี้ครอบคลุมโครงสร้างเดิมทั้งหมด พร้อมตัวอย่างการขยายฟังก์ชันโดยไม่เปลี่ยนโมเดล SdUser